

From Engine Evaluations to Human Explanations: A Multi-Level LLM Pipeline for Critical Chess Positions

Bachelor Semester Project S2 (Academic Year 2025/26), University of Luxembourg

Viktor Tsvetkov

University of Luxembourg

Esch-sur-Alzette, Luxembourg

viktor.tsvetkov.001@student.uni.lu

ABSTRACT

Chess engines can say *who* is better in a position with great precision, but the number they output explains nothing, and the same explanation is not equally useful to a beginner and to a master. This project builds and evaluates a pipeline that turns an engine-evaluated chess position into three explanations, each tailored to a different playing strength (under 1400, 1400–2200, and 2200+ Elo). The pipeline rests on three inputs per position: the position itself, its engine evaluation, and a piece-by-piece attribution computed with the *chessplainer* library, which uses SHAP to measure every piece’s contribution to White’s winning probability. A concept library of 207 chess terms, reconstructed from the concepts used to probe AlphaZero, serves as the controlled vocabulary. I applied the pipeline to a dataset of 25 critical positions from games of Fischer and Tal, verified the dataset mechanically (every recorded move replays exactly; the stored evaluations agree with fresh Stockfish analysis), issued one automated LLM query per position, and organised the output as a results file with one row per position and one column per audience. Every explanation was then scored on a 1–5 Likert scale along four dimensions: groundedness, clarity, faithfulness, and cross-level coherence, ten values per record. The audit process itself proved valuable: checking the texts against the SHAP attributions exposed a genuine factual error (a bishop pair credited to the wrong side) that reading the text alone had not revealed.

KEYWORDS

chess, explainable AI, SHAP, large language models, audience-adaptive explanation, Likert evaluation

1 INTRODUCTION

Modern chess engines evaluate positions far better than any human, but they communicate in a single number. An evaluation of +1.3 tells a strong player a great deal and a beginner almost nothing; conversely, an explanation written for a beginner (“do not lose your pieces”) wastes a master’s time. Explanation is not only about being correct, it is about being correct *for someone*.

This Bachelor Semester Project asks whether a large language model (LLM), given the right structured inputs, can explain one and the same chess position three times: for a beginner, for a club player, and for a master-level reader. The work is practical and pipeline-shaped. I do not train a model and I do not try to improve the engine. Instead I study the layer between the engine and the human: what information the LLM must receive so that its explanations are *grounded* (true on the board), *faithful* (crediting the factors that

actually carry the position), *clear* for the intended audience, and *coherent* across the three audiences.

The project produced five things: a concept library of 207 chess terms serving as a fixed vocabulary; a prompt that defines the three audiences and the rules for reading an engine score; a mechanically verified dataset of 25 critical positions; an automated pipeline in which the *chessplainer* library [4] computes a SHAP-based piece-by-piece attribution for every position and a script issues one LLM query per position; and a ten-value Likert evaluation of every record, with a written rating guide. The main methodological finding mirrors a lesson from evaluation research in general: the audit is not bureaucracy. Checking each explanation against the position’s actual attribution caught a real factual error that casual reading had missed.

2 BACKGROUND AND PROJECT CONTEXT

2.1 Engine evaluations and chess concepts

A chess engine such as Stockfish [6] reports a position’s evaluation in centipawns from White’s point of view: +100 means an advantage worth about one pawn for White, negative values favour Black, and a forced checkmate appears as an extreme spike far outside the normal range. The evaluation is the ground truth of this project, but it is opaque: it does not say *which* pieces or features produce it.

McGrath et al. [1] studied what AlphaZero [3] knows about chess by probing its network for human chess concepts, drawn from the components of Stockfish 8’s evaluation function plus custom concepts of their own. Their concept set is the vocabulary of this project (Section 3): if an explanation talks about mobility, king safety, or passed pawns, it should use those words in a fixed, checkable sense.

2.2 Explaining a position piece by piece

Knowing that White is better is not the same as knowing *why*. The *chessplainer* library [4] answers the *why*-question at the level of pieces with SHAP [2], a standard attribution method. The idea is masking: remove subsets of pieces from the board (kings always stay), let Stockfish evaluate each masked board, convert the scores to win/loss probabilities, and let the SHAP kernel assign every piece a value: its contribution to White’s probability of winning. A piece with +0.30 raises White’s winning chances by roughly thirty percentage points through its presence; a negative piece works for Black. Concretely, a position with n non-king pieces is encoded as a binary vector of length n ; a masking function turns any such vector into a board that keeps exactly the selected pieces (kings always remain, and boards that become illegal are handled by flipping the

side to move or scored as unknown). Stockfish evaluates every masked board, the score is converted to win/draw/loss probabilities under the Lichess WDL model, and the SHAP kernel explainer fits attributions over the sampled coalitions, yielding for every piece its share of White’s winning probability. These attributions are model-faithful — they measure what the engine actually computes, not what a rule of thumb says — and they are the third input of every explanation query.

2.3 Scope

The pipeline explains the position *before* the critical move of each record, paired with that position’s own evaluation, so that position and number always belong together. Move recommendation, opening theory, and interactive coaching are out of scope. The LLM is treated as a component to be constrained and measured, not as an oracle.

3 THE CONCEPT LIBRARY

The first deliverable was a machine-readable library of the concepts in [1], reconstructed from the paper’s supplementary tables. This was less mechanical than it sounds. The paper’s Table S1 lists only 13 Stockfish evaluation *families*, yet the paper states it used 93 Stockfish concepts. The resolution is an expansion scheme: each family exists in up to three *sides* (mine, opponent, total) and three *game phases* (middlegame, endgame, phase-weighted). Four families (material, imbalance, pawns, total) exist only as totals, nine exist per side, and $4 \times 3 + 9 \times 3 \times 3 = 93$ — exactly the paper’s number. I verified the family structure directly against Stockfish 8’s source code (`evaluate.cpp`), whose evaluation trace prints precisely those 13 terms and reports exactly those four families as totals only.

Enumerating the custom concepts of Tables S2–S3 with the same care gives 114 fully named concepts (the paper claims 116; the two-concept gap traces to side suffixes that the printed tables leave ambiguous, which I documented rather than invented). The released library therefore contains 207 explicitly enumerated concepts as `concept_name`, `description` pairs — for example `mobility_mine_mg` (the side to move’s weighted mobility in the middlegame) or `has_connected_rooks` — and is prepended to every LLM query as its controlled vocabulary. One subtlety worth recording: material appears twice in the source tables with different definitions (Stockfish’s phase-dependent score and a simple 1/3/3/5/9 count); the library keeps both and documents the difference rather than silently merging them. The point of a controlled vocabulary is exactly this kind of bookkeeping: when an explanation says “mobility,” reviewer and model must mean the same measurable thing.

4 DATASET AND VERIFICATION

The dataset holds 25 critical positions extracted from historical games of Robert J. Fischer and Mikhail Tal (1949–1959) against various opponents. Each record stores the position before and after the critical move (as FEN), the move itself, and the engine evaluation before and after, in White-point-of-view centipawns. Table 1 summarises the composition.

Table 1: The dataset at a glance.

Records (one critical position each)	25
White to move / Black to move	11 / 14
Middlegames / endgames	14 / 11
Evaluation range before the move (cp)	−111 to +1052
Forced-mate spikes (after the move)	2
Games by	Fischer, Tal
Years	1949–1959

Table 2: Stored evaluations vs. fresh Stockfish 16 (depth 16); centipawns, White’s point of view.

Rec.	Dataset	SF16	Rec.	Dataset	SF16
1	+129	+126	18	+7	+19
3	+707	+641	20	+67	+56
8	−111	−135	22	+168	+183
13	−96	−117	25	+547	+567

Table 3: The three audiences.

Tier	Elo	Register
Beginner	< 1400	everyday words, no jargon
Intermediate	1400–2200	standard chess terms
Advanced	2200+	full technical vocabulary

Because “consistency between the evaluation and the position” is the whole foundation, I verified the dataset mechanically before using it. First, *replay consistency*: for every record I parsed the position, applied the recorded move with the python-chess library [5], and compared the result with the stored after-position. All 25 records reproduce their after-FEN exactly. Second, *evaluation consistency*: I re-analysed a sample of eight positions with a fresh Stockfish 16 at depth 16 and compared (Table 2). The stored values are genuine engine evaluations of exactly these positions — close in magnitude and agreeing on the side of the advantage in 8 of 8 samples. The differences of a few dozen centipawns are what one expects between engine versions and search depths; what matters is that no stored number contradicts the position it is attached to.

Third, *mate encoding*: forced mates appear in this dataset as evaluation spikes on the order of $\pm 100,000$ (for example $1052 \rightarrow 99989$ in record 7 and $-96 \rightarrow -99995$ in record 13). Both spikes occur only *after* the critical move; no starting position is itself a forced mate, so no explanation may claim one.

5 THE MULTI-LEVEL EXPLAINER

The heart of the system is a prompt that fixes the task, the audiences, and the rules. The LLM receives the concept library as standing context and then three tagged inputs per query: the position (FEN), the evaluation, and the piece-by-piece attribution.

Three rules matter most. *Reading the number*: the evaluation is White-point-of-view; the prompt gives a verbal scale (roughly equal below 0.3 pawns, slight edge to 0.9, clear advantage to 1.5, large

advantage to 3, winning above) and defines a forced mate not by notation but by the evaluation spiking far outside the normal range — matching how the dataset actually encodes mates. *Length*: each explanation is at most 3–4 sentences, with a longer form permitted only for a forced mate. *Grounding*: every claim must be supported by the three inputs and the library; inventing moves or evaluations is forbidden, and the three explanations must agree on who is better — they may differ in depth and vocabulary, never in the verdict. The output format is a small CSV so that the answers can be parsed mechanically: the model must return one header row and one data row with the three explanations, every field quoted. The querying script parses that reply with a standard CSV reader and tolerates minor header variations, so a formatting wobble in a single answer cannot corrupt the results file.

6 PIECE-BY-PIECE ATTRIBUTION

The piece-by-piece input went through three designs, and the failures are as instructive as the final choice.

The first design used classical piece-square tables: every piece receives the positional value of its square. This is transparent and instant, but it is a rule of thumb — it cannot see tactics and it values an isolated pawn like a healthy one. The second design tried leave-one-out with the engine’s neural evaluation directly (remove a piece, re-evaluate, subtract material). The numbers were noisy and sometimes absurd: a dangerous black queen could appear to *favour White*, because a neural evaluation does not decompose linearly. The lesson: an attribution has to be built on a sound method, not improvised.

The final design uses *chessplainer* as intended. Its *EngineWrapper* turns Stockfish into a probabilistic classifier (win/loss probability for White under the Lichess win-draw-loss model), *mask_chessboard* generates boards with subsets of pieces, and a *SHAP KernelExplainer* assigns each non-king piece its contribution to White’s winning probability. I ran it on all 25 positions with a fixed random seed, engine depth 14 for the base position and depth 8 for masked boards; the full run takes about 145 seconds (≈ 6 s per position). Two artefacts are produced: a full per-piece table (all pieces, both SHAP outputs) and a compact textual line per position — the eight largest contributions by magnitude, e.g. for record 24:

Table 4: chessplainer attribution for record 24 (top 8 pieces; + favours White).

Piece	SHAP	Piece	SHAP
Rc7 (W)	+0.428	Nf7 (B)	−0.183
Ba8 (B)	−0.236	a4 (W)	+0.164
Bf8 (B)	−0.202	f4 (B)	−0.129
Nd2 (W)	+0.185	g2 (W)	+0.107

The attribution reads like a story: White’s seventh-rank rook is the dominant piece; Black’s two bishops and knight matter for Black but less; White’s passed pawns contribute. This story became the target that every explanation had to match.

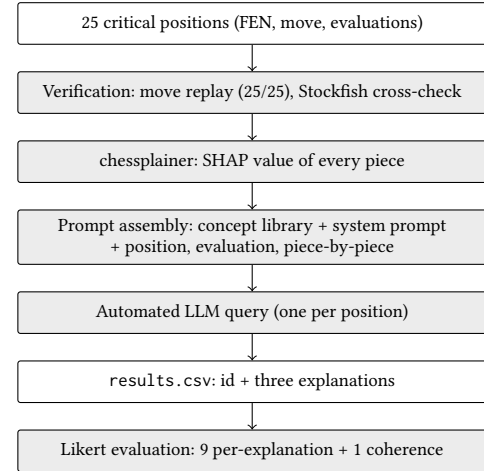


Figure 1: The pipeline. Plain boxes are artefacts; shaded boxes are steps.

7 AUTOMATED QUERYING AND RESULTS

The querying is fully scripted; nothing was pasted by hand. The batch script loads the dataset, computes (or reuses) the chessplainer attribution for each position, assembles the standing context and the three tagged inputs, sends exactly one query per position — 25 queries in total — and parses the small CSV each answer contains. The API key is read from an environment variable only, so the published script contains no secret. A dry-run mode prints the exact inputs of any record without spending a query, which is also how the assembled prompts were inspected during development.

The deliverable results file has exactly four columns: id, and one explanation per audience. To give the flavour, record 24’s beginner explanation begins “White is winning this endgame. White’s rook has broken through to c7, deep in Black’s position, there’s a dangerous pawn on b6 pushing toward promotion, and Black’s two bishops sit passively in the corners...”, while the advanced tier calls the same situation “a rook-versus-minor-pieces imbalance where the rook’s activity decides.” All 75 explanations respect the sentence cap, and a scripted check confirms that the three tiers of every record state the same verdict.

A worked example

Record 24 (Tal–Visockis, an endgame evaluated at +405 centipawns) shows the register shift the tiers are designed for. The beginner text reads: “White is winning this endgame. White’s rook has broken through to c7, deep in Black’s position, there’s a dangerous pawn on b6 pushing toward promotion, and Black’s two bishops sit passively in the corners. White should push the pawn and let the rook dominate.” The advanced text describes the same truth in a different language: “White is winning ($\sim +4$): the c7-rook rules the seventh and, with the b6 passer, paralyzes Black’s uncoordinated a8/f8-bishops and f7-knight — a rook-versus-minor-pieces imbalance where the rook’s activity decides. White pushes the passer while creating a second front. Black has no counterplay.” Both credit the pieces that Table 4 actually ranks on top; what changes is vocabulary and depth, not the verdict.

Table 5: Rating-guide anchors for *faithfulness* (the other dimensions are defined analogously in the released guide).

Score	Anchor
5	Headlines the attribution’s top contributors.
4	Mostly aligned; one top factor omitted or a secondary factor foregrounded.
3	Partially aligned with the attribution.
2	Credits the wrong factors.
1	The story contradicts the attribution.

8 EVALUATION

8.1 Dimensions and rating guide

Each record receives ten integer scores on a 1–5 Likert scale [7]: *groundedness*, *clarity*, and *faithfulness* for each of the three explanations (nine values), plus one global *coherence* score that considers all three at once. Groundedness asks whether the explanation reports things that are true in the starting position; clarity whether it reads well for its tier; faithfulness whether what is claimed as the explanation is the *actual* explanation — whether the pieces the text credits match the SHAP attribution and the evaluation; coherence whether the three explanations agree. A written rating guide defines anchors for every score; Table 5 shows the faithfulness anchors as an example, and every score below 5 is recorded together with its reason in a notes file, so that no deduction is unexplained.

The scoring method deserves a sentence of honesty. Scores were assigned by manual annotation, but against machine-verified evidence rather than impression: groundedness against the board and the (Stockfish-cross-checked) evaluations, faithfulness against the computed attributions, and coherence additionally checked by a script that parses the verdict of every tier. One person did all the rating.

8.2 Results

Table 6 shows the per-dimension means over the 25 records. Of the 250 assigned values, 236 are 5 and 14 are 4; no value fell below 4. The deductions concentrate where one would expect: nine of the fourteen are clarity deductions, almost all on the advanced tier, whose compressed technical style produces dense multi-clause sentences; three are faithfulness deductions on beginner texts that, in simplifying, omit a top contributor the attribution flags (for example the e2-bishop in record 18); two are groundedness deductions for phrases that overreach the verified facts (“perfectly placed”; an inferred detail about the shape of a mating attack). Coherence is 5 throughout, which the scripted agreement check corroborates.

8.3 What the audit caught

The high averages are the *end* of the story, not the whole of it. The faithfulness audit — systematically comparing every text against its position’s attribution — caught a genuine factual error in an earlier draft of record 24: the text credited White with “the two bishops,” but the attribution showed both bishops with negative (Black-favouring) values, and re-checking the board confirmed that

Table 6: Likert means per dimension (25 records; 1–5 integers).

	Groundedness	Clarity	Faithfulness
Beginner	4.96	5.00	4.88
Intermediate	5.00	4.88	5.00
Advanced	4.96	4.76	5.00
Coherence (global)		5.00	

the a8- and f8-bishops are Black’s. The plausible-sounding sentence had survived ordinary proofreading; the attribution exposed it in seconds. The corrected text now tells the true story (White’s seventh-rank rook and passed pawn against Black’s passive bishop pair), and the episode is the project’s clearest evidence that a faithfulness check against a computed attribution is a working debugging tool, not a formality.

A second, structural observation: SHAP attributions are dominated by heavy pieces, because removing a queen changes the winning probability far more than removing a pawn. Pedagogically salient features — a passed pawn one step from promotion — rank below the queens and rooks even when they decide the game (record 10’s c7-pawn scores +0.22, behind both rooks). Faithful explanations therefore need to weave the attribution’s headline pieces *and* the human-salient features together, which is exactly what the tier design is for: the beginner hears about the pawn about to promote, the master hears which pieces carry the winning probability, and both are true.

9 DISCUSSION AND LIMITATIONS

The central lesson is that in an explanation pipeline the evaluation machinery earns its keep. Replay verification guaranteed that positions, moves, and evaluations belong together; the Stockfish cross-check guaranteed the numbers are real; the attribution audit turned “does this text sound right?” into “does this text match what the engine actually computes?” — and it is the audit, not the writing, that found the one real factual error.

A second lesson concerns *what* the scores measure. Because the faithfulness audit drove a correction round before scoring, the Likert values describe the released artifact — the pipeline including its quality control — rather than raw model output. Both are legitimate objects of study, but they answer different questions, and a report should say which one it is answering; this one evaluates the artifact. The coherence dimension illustrates the same care from the other side: cross-tier agreement can be checked by a script (and was), which is why a perfect coherence column is a verified property here rather than an optimistic self-grade.

Several limits apply. The dataset is small (25 positions) and drawn from two players’ games, so the score averages describe this set, not chess explanation in general. One person did all the Likert rating, so inter-rater agreement was not measured, and the scores were assigned to the *final* texts — after the faithfulness audit had already driven one round of corrections — so they describe the released artifact rather than first-pass LLM output. The released explanations were produced by the assistant model working directly from the assembled pipeline inputs rather than captured through the API

script; the script reproduces the queries exactly and is part of the supplementary material, but regenerating byte-identical text is not guaranteed, since models vary between runs. Finally, the SHAP values are computed at modest engine depth with a fixed seed for reproducibility; deeper analysis would shift individual numbers, though hardly the ranking of the top pieces.

10 CONCLUSION AND FUTURE WORK

This project built and evaluated a complete pipeline from an engine-evaluated chess position to three audience-tailored explanations: a 207-concept controlled vocabulary reconstructed and verified against Stockfish 8's own source; a mechanically verified 25-position dataset; SHAP piece attributions computed with chessplainer for every position; fully scripted LLM querying (one query per position, no keys in the code); a four-column results file; and a ten-value Likert evaluation per record with a written rating guide. The headline numbers are high (236 of 250 values at 5), but the more useful products are the audit findings: a faithfulness check against computed attributions catches factual errors that reading misses, and heavy-piece dominance in SHAP means faithful and pedagogically useful are not automatically the same thing.

The natural next steps: score first-pass LLM output (before any correction loop) to measure the model rather than the artifact; add a second rater and an agreement statistic; scale beyond 25 positions; and close the loop by feeding the evaluation findings back into the prompt — for example, an explicit instruction to name the attribution's top contributor in every tier.

SELF-EVALUATION

This project taught me that the distance between “a script that works” and “a pipeline you can trust” is verification. The parts I am most satisfied with are the ones nobody sees in the output: replaying all 25 moves against the stored positions, cross-checking the evaluations against a fresh engine, and auditing every explanation against its SHAP attribution — the step that caught a factual error I had read past several times. I also learned to let go of my own solutions: my first two piece-by-piece designs (piece-square tables, then naive leave-one-out) were instructive failures, and adopting the purpose-built chessplainer library gave a sounder foundation than either. Working across the whole chain — paper concepts, engine internals, SHAP, prompt design, and evaluation rubrics — showed me how much of “explainable AI” is really careful bookkeeping about what is true, for whom, and how you would know.

REFERENCES

- [1] T. McGrath, A. Kapishnikov, N. Tomašev, A. Pearce, M. Wattenberg, D. Hassabis, B. Kim, U. Paquet, and V. Kramnik. 2022. Acquisition of Chess Knowledge in AlphaZero. *PNAS* 119, 47 (2022).
- [2] S. M. Lundberg and S.-I. Lee. 2017. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*.
- [3] D. Silver et al. 2018. A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play. *Science* 362, 6419 (2018), 1140–1144.
- [4] F. Spinnato. chessplainer: Explaining Chess Positions Using SHAP. <https://github.com/fspinna/chessplainer>.
- [5] N. Fiekas. python-chess: A Chess Library for Python. <https://github.com/niklasf/python-chess>.
- [6] The Stockfish Developers. Stockfish: A Free and Strong UCI Chess Engine. <https://stockfishchess.org>.
- [7] R. Likert. 1932. A Technique for the Measurement of Attitudes. *Archives of Psychology* 22, 140 (1932), 1–55.